NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD

PARALLEL AND DIST COMP. (LAB)

Lab Report: 02

Submitted to
Mr. Farhad M. Riaz

Submitted By
Aleeha Zahid
(BSAI-136)

Task 1: Basic Parallel For Loop

This uses OpenMP to illustrate a basic parallel for loop. The code has a loop that assigns a separate thread to each of the 20 iterations. Though not always in sequential order, the output displays how the iterations are split up among the available threads.

CODE :

main.cpp

```cpp
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
    int N = 20;
    #pragma omp parallel for
        for ( int i = 0; i < N; i++ ){
            printf("Iteration %d is executed by thread %d\n", i, omp_get_thread_num());
        }
    return 0;
}

```

OUTPUT:

```
Iteration 0 is executed by thread 0
Iteration 1 is executed by thread 0
Iteration 2 is executed by thread 0
Iteration 16 is executed by thread 6
Iteration 3 is executed by thread 1
Iteration 4 is executed by thread 1
Iteration 5 is executed by thread 1
Iteration 12 is executed by thread 4
Iteration 13 is executed by thread 4
Iteration 6 is executed by thread 2
Iteration 7 is executed by thread 2
Iteration 8 is executed by thread 2
Iteration 14 is executed by thread 5
Iteration 15 is executed by thread 5
Iteration 9 is executed by thread 3
Iteration 10 is executed by thread 3
Iteration 11 is executed by thread 3
Iteration 17 is executed by thread 6
Iteration 18 is executed by thread 7
Iteration 19 is executed by thread 7


...Program finished with exit code 0
Press ENTER to exit console.
```

Task 2: Parallel For Loop with Thread Count

This code clearly indicates how many threads should be used. The output displays how the 20 iterations were divided across the 10 threads, with each thread managing a different section of the loop.

CODE:

main.cpp

```cpp
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
    int N = 20;
    #pragma omp parallel for num_threads(10)
        for ( int i = 0; i < N; i++ ){
            printf("Iteration %d is executed by thread %d\n", i, omp_get_thread_num());
        }
    return 0;
}

```

OUTPUT:

```
Iteration 4 is executed by thread 2
Iteration 5 is executed by thread 2
Iteration 2 is executed by thread 1
Iteration 3 is executed by thread 1
Iteration 18 is executed by thread 9
Iteration 19 is executed by thread 9
Iteration 8 is executed by thread 4
Iteration 9 is executed by thread 4
Iteration 14 is executed by thread 7
Iteration 15 is executed by thread 7
Iteration 12 is executed by thread 6
Iteration 13 is executed by thread 6
Iteration 16 is executed by thread 8
Iteration 17 is executed by thread 8
Iteration 0 is executed by thread 0
Iteration 1 is executed by thread 0
Iteration 6 is executed by thread 3
Iteration 7 is executed by thread 3
Iteration 10 is executed by thread 5
Iteration 11 is executed by thread 5


...Program finished with exit code 0
Press ENTER to exit console.
```

Task 3: Parallel Summation with a Race Condition

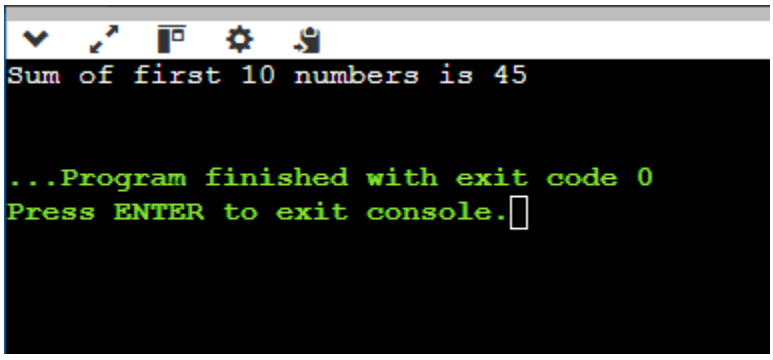
This finds the sum of the first ten numbers. Several threads attempt to update the sum variable at the same time so the sum += i action generates a race condition. Due to a limited number of repetitions, the output displays the correct sum of 45 but in a more complicated situation, this method would probably produce an incorrect sum.

CODE:

main.cpp

```cpp
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
    int sum = 0;
    int n = 10;
    #pragma omp parallel for
        for ( int i = 0; i < n; i++ ){
            sum += i;
        }
    printf("Sum of first %d numbers is %d\n", n, sum);
    return 0;
}
```

OUTPUT:



Task 4: Parallel Summation with Local and Global Variables

In this we write local sum for every thread and then combine them into a global sum for proper method of performing parallel summing. To avoid a race issue, each thread computes its own partial total for the iterations it is given. The sum of these local sums gives the ultimate value, which for the first 20 numbers is 210.

CODE:

main.cpp

```cpp
#include <iostream>
#include <stdio.h>
#include <omp.h>

int main ( ) {
    int n =20;

    #pragma omp parallel
    {
      int l_sum = 0;
      int thread_id = omp_get_thread_num();

      #pragma omp for
        for (int i = 0; i <= n; i ++){
            l_sum += i;
        }
    printf("thread is : %d and local sum is : %d\n",thread_id,l_sum);
    }

    int g_sum = 0;
    for (int i =0;i <= n; i++)
    {
        g_sum += i;
    }
    printf("final sum of first %d numbers is : %d\n",n,g_sum);

return 0;
}
```

OUTPUT:

```
thread is : 4 and local sum is : 39
thread is : 0 and local sum is : 3
thread is : 1 and local sum is : 12
thread is : 6 and local sum is : 35
thread is : 5 and local sum is : 31
thread is : 7 and local sum is : 39
thread is : 3 and local sum is : 30
thread is : 2 and local sum is : 21
final sum of first 20 numbers is : 210


...Program finished with exit code 0
Press ENTER to exit console.
```

Task 5: Parallel Sections

This task divides various pieces of code among the available threads using the **#pragma omp sections** directive. This method assigns a thread to each **#pragma omp section** rather than iterating through a loop so that a distinct thread is used to complete each task.

CODE:

```
#include <iostream>
#include <stdio.h>
#include <omp.h>

int main () {
    #pragma omp parallel
    {
        #pragma omp sections
        {
            #pragma omp section
            {
                printf("Task 1 executed by thread %d\n",omp_get_thread_num());
            }
            #pragma omp section
            {
                printf("Task 2 executed by thread %d\n",omp_get_thread_num());
            }
            #pragma omp section
            {
                printf("Task 3 executed by thread %d\n",omp_get_thread_num());
            }
        }
    }
return 0;
}

```

OUTPUT:

```
Task 1 executed by thread 6
Task 2 executed by thread 7
Task 3 executed by thread 5


...Program finished with exit code 0
Press ENTER to exit console.
```

Task 6: Parallel Sections for Sum and Product

This code calculates the sum and the product of numbers in parallel using parallel sections. The product is calculated by one thread, and the sum is calculated by another. The output displays the output of both as well as their thread IDs.

CODE:

```cpp
#include <iostream>
#include <stdio.h>
#include <omp.h>

int main () {
    int sum = 0;
    int product = 1;
    #pragma omp parallel
    {
        #pragma omp sections
        {
            #pragma omp section
            {
               for (int i =1;i<=5;i++)
               {
                   sum += i;
               }
            printf("The sum is %d and thread id is %d\n",sum,omp_get_thread_num());
            }
            #pragma omp section
            {
               for (int i =1;i<=5;i++)
               {
                   product *= i;
               }
            printf("The product is %d and thread id is %d\n",product,omp_get_thread_num());

            }
        }
    }
return 0;
}
```


OUTPUT:

```
The sum is 15 and thread id is 3
The product is 120 and thread id is 7


...Program finished with exit code 0
Press ENTER to exit console.
```

Task 7: Parallel Sections for Min and Max

This code finds an array's minimum and maximum values using parallel sections. Each task is carried out by a different thread, and the output accurately indicates that the smallest value is 3 and the maximum is 56.

CODE:

```c
#include <stdio.h>
#include <omp.h>

int main () {
    int array[] = {56,12,3,44,5};
    int min = array[0];
    int max = array[0];
    #pragma omp parallel
    {
        #pragma omp sections
        {
            #pragma omp section
            {
             for (int i =0;i<5;i++)
             {
                 if (array[i] < min)
                 {
                     min = array[i];
                 }
             }
            printf("The minimum element is %d\n",min);
            }
            #pragma omp section
            {
             for (int i =0;i<5;i++)
             {
                 if (array[i] > max)
                 {
                     max = array[i];
                 }
             }
            printf("The maximum element is %d\n",max);
            }
        }
    }
return 0;
}
```

OUTPUT:

```
The minimum element is 3
The maximum element is 56


...Program finished with exit code 0
Press ENTER to exit console.
```